Mini Compiler Project

CSE 430 Compiler Design Lab

Submitted by: Basirul Hasin
ID: 21101045
Department of Computer Science and Engineering
University of Asia Pacific

Contents

1 Introduction

This report details the preparation and implementation of a mini compiler using Python and PLY (Python Lex-Yacc). The project focuses on compiling a simplified C-like language with variables, arithmetic, arrays, loops, conditionals, floating-point, and function support. Each compiler phase — lexical analysis, parsing, symbol table, intermediate code (TAC) — is demonstrated with logs and outputs.

2 Design

The compiler includes the following fundamental phases:

- **Lexical Analysis:** Tokenizes the code (identifiers, literals, keywords, symbols).
- **Parsing:** Checks code structure with grammar, supporting control flow, arrays, and functions.
- **Symbol Table:** Manages variable/function declarations and values, supports nested scope.
- **Intermediate Code Generation:** Produces Three Address Code for all operations.
- **Error Detection:** Prints syntax and semantic errors.

Project Flow Diagram:

Source Code
↓
Lexical Analysis (Tokens)
↓
Parsing (Syntax, Grammar)
↓
Symbol Table Handling
↓
Intermediate Code Generation (TAC)
↓
Output/Diagnostics

3 Implementation

```
1 import ply.lex as lex
2 import ply.yacc as yacc
3
4 tokens = (
5     'ID', 'INT', 'FLOAT', 'NUMBER', 'FNUMBER', 'PLUS', 'MINUS', 'TIMES'
          , 'DIVIDE',
6     'EQUALS', 'LPAREN', 'RPAREN', 'LBRACE', 'RBRACE', 'SEMICOLON', '
          COMMA',
7     'IF', 'ELSE', 'WHILE', 'PRINT', 'RETURN',
```

```
    'LSQUARE', 'RSQUARE',
    'EQ', 'NE', 'LT', 'GT', 'LE', 'GE'
)

t_PLUS = r'\+'
t_MINUS = r'-'
t_TIMES = r'\*'
t_DIVIDE = r'/'
t_EQUALS = r'='
t_LPAREN = r'\('
t_RPAREN = r'\)'
t_LBRACE = r'\{'
t_RBRACE = r'\}'
t_SEMICOLON = r';'
t_COMMA = r','
t_LSQUARE = r'\['
t_RSQUARE = r'\]'
t_EQ = r'=='
t_NE = r'!='
t_LE = r'<='
t_GE = r'>='
t_LT = r'<'
t_GT = r'>'

reserved = {
    'if': 'IF', 'else': 'ELSE', 'while': 'WHILE', 'print': 'PRINT',
    'return': 'RETURN', 'int': 'INT', 'float': 'FLOAT'
}

def t_FNUMBER(t):
    r'\d+\.\d+'
    t.value = float(t.value)
    return t

def t_NUMBER(t):
    r'\d+'
    t.value = int(t.value)
    return t

def t_ID(t):
    r'[a-zA-Z_][a-zA-Z0-9_]*'
    t.type = reserved.get(t.value, 'ID')
    return t

t_ignore = ' \t\r'

def t_newline(t):
    r'\n+'
    t.lexer.lineno += len(t.value)

def t_comment(t):
    r'//.*'
    pass

def t_error(t):
    print(f"Lexical error: '{t.value[0]}' at line {t.lexer.lineno}")
    t.lexer.skip(1)

```

```
lexer = lex.lex()

class SymbolTable:
    def __init__(self):
        self.stack = [{}]
    def push(self):
        self.stack.append({})
    def pop(self):
        self.stack.pop()
    def declare(self, name, value=None):
        self.stack[-1][name] = value
    def assign(self, name, value):
        for scope in reversed(self.stack):
            if name in scope:
                scope[name] = value
                return
        self.stack[-1][name] = value
    def lookup(self, name):
        for scope in reversed(self.stack):
            if name in scope:
                return scope[name]
        return None
    def __repr__(self):
        return str(self.stack)

symbol_table = SymbolTable()
function_table = {}
tac = []
_label = [0]
_temp = [0]

def new_label():
    _label[0] += 1
    return f"L{_label[0]}"

def new_temp():
    _temp[0] += 1
    return f"t{_temp[0]}"

class Array:
    def __init__(self, size):
        self.size = size
        self.val = [0] * size

precedence = (
    ('left','EQ','NE','LT','LE','GT','GE'),
    ('left','PLUS','MINUS'), ('left','TIMES','DIVIDE')
)

def p_program(p):
    '''program : items'''
    print("\nSYMBOL TABLE:", symbol_table)
    print("\n--- Three Address Code ---")
    for line in tac:
        print(line)

def p_items(p):
    '''items : items item
```

```
␣␣␣␣␣␣␣␣␣␣␣␣␣|␣item'''

def p_item(p):
    '''item␣:␣decl
␣␣␣␣␣␣␣␣␣␣␣␣|␣func
␣␣␣␣␣␣␣␣␣␣␣␣|␣stmt'''

def p_decl(p):
    '''decl␣:␣type␣varlist␣SEMICOLON'''

def p_type(p):
    '''type␣:␣INT
␣␣␣␣␣␣␣␣␣␣␣␣|␣FLOAT'''
    p[0] = p[1]

def p_varlist(p):
    '''varlist␣:␣varlist␣COMMA␣var
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣|␣var'''

def p_var(p):
    '''var␣:␣ID
␣␣␣␣␣␣␣␣␣␣␣␣|␣ID␣LSQUARE␣NUMBER␣RSQUARE'''
    if len(p) == 2:
        symbol_table.declare(p[1])
    else:
        symbol_table.declare(p[1], Array(p[3]))

def p_func(p):
    'func␣:␣type␣ID␣LPAREN␣params␣RPAREN␣LBRACE␣funcbody␣RBRACE'
    function_table[p[2]] = (p[4], p[7])
    tac.append(f"#␣func␣{p[2]}({',␣'.join(str(x)␣for␣x␣in␣p[4])})")

def p_params(p):
    '''params␣:␣paramlist
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣|␣empty'''
    p[0] = p[1] if p[1] is not None else []

def p_paramlist(p):
    '''paramlist␣:␣paramlist␣COMMA␣param
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣|␣param'''
    if len(p) == 2:
        p[0] = [p[1]]
    else:
        p[0] = p[1] + [p[3]]

def p_param(p):
    'param␣:␣type␣ID'
    p[0] = p[2]

def p_funcbody(p):
    '''funcbody␣:␣funcstmts␣func_return'''
    p[0] = (p[1], p[2])

def p_funcstmts(p):
    '''funcstmts␣:␣funcstmts␣stmt
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣|␣empty'''

def p_func_return(p):
```

```
    '''func_return : RETURN expr SEMICOLON
                   | empty'''
    if len(p) == 4:
        tac.append(f"RET {p[2]['place']}")

def p_stmt_assign(p):
    'stmt : ID EQUALS expr SEMICOLON'
    symbol_table.assign(p[1], p[3]['value'])
    tac.append(f"{p[1]} = {p[3]['place']}")

def p_stmt_assign_arr(p):
    'stmt : ID LSQUARE expr RSQUARE EQUALS expr SEMICOLON'
    arr = symbol_table.lookup(p[1])
    if not isinstance(arr, Array):
        print(f"Semantic error: '{p[1]}' not array")
    else:
        idx = p[3]['value']
        arr.val[idx] = p[6]['value']
        tac.append(f"{p[1]}[{idx}] = {p[6]['place']}")

def p_stmt_print(p):
    'stmt : PRINT LPAREN expr RPAREN SEMICOLON'
    print(f"OUTPUT: {p[3]['value']}")
    tac.append(f"print {p[3]['place']}")

def p_stmt_if_else(p):
    'stmt : IF LPAREN expr RPAREN LBRACE block RBRACE else_block'

def p_else_block(p):
    '''else_block : ELSE LBRACE block RBRACE
                  | empty'''

def p_block(p):
    '''block : stmts
             | empty'''

def p_stmts(p):
    '''stmts : stmts stmt
             | stmt'''

def p_stmt_while(p):
    'stmt : WHILE LPAREN expr RPAREN LBRACE block RBRACE'

def p_stmt_func_call(p):
    'stmt : ID LPAREN call_args RPAREN SEMICOLON'
    tac.append(f"call {p[1]}({', '.join(p[3])})")

def p_call_args(p):
    '''call_args : call_args COMMA expr
                 | expr
                 | empty'''
    if len(p) == 2 and p[1] is not None:
        p[0] = [str(p[1]['place'])]
    elif len(p) == 4:
        p[0] = p[1] + [str(p[3]['place'])]
    else:
        p[0] = []

```

```
240 def p_expr_binop(p):
241     '''expr : expr PLUS expr
242             | expr MINUS expr
243             | expr TIMES expr
244             | expr DIVIDE expr
245             | expr EQ expr
246             | expr NE expr
247             | expr LT expr
248             | expr GT expr
249             | expr LE expr
250             | expr GE expr'''
251     left, right = p[1], p[3]
252     temp = new_temp()
253     tac.append(f"{temp} = {left['place']} {p[2]} {right['place']}")
254     if p[2] == '+':
255         value = left['value'] + right['value']
256     elif p[2] == '-':
257         value = left['value'] - right['value']
258     elif p[2] == '*':
259         value = left['value'] * right['value']
260     elif p[2] == '/':
261         value = left['value'] / right['value']
262     elif p[2] == '==':
263         value = int(left['value'] == right['value'])
264     elif p[2] == '!=':
265         value = int(left['value'] != right['value'])
266     elif p[2] == '<':
267         value = int(left['value'] < right['value'])
268     elif p[2] == '>':
269         value = int(left['value'] > right['value'])
270     elif p[2] == '<=':
271         value = int(left['value'] <= right['value'])
272     elif p[2] == '>=':
273         value = int(left['value'] >= right['value'])
274     p[0] = {'place': temp, 'value': value}
275
276 def p_expr_fnumber(p):
277     'expr : FNUMBER'
278     p[0] = {'place': str(p[1]), 'value': p[1]}
279
280 def p_expr_number(p):
281     'expr : NUMBER'
282     p[0] = {'place': str(p[1]), 'value': p[1]}
283
284 def p_expr_id(p):
285     'expr : ID'
286     val = symbol_table.lookup(p[1])
287     if val is None:
288         print(f"Semantic error: '{p[1]}' used before assignment.")
289         val = 0
290     p[0] = {'place': p[1], 'value': val}
291
292 def p_expr_arr(p):
293     'expr : ID LSQUARE expr RSQUARE'
294     arr = symbol_table.lookup(p[1])
295     idx = p[3]['value']
296     if isinstance(arr, Array):
297         v = arr.val[idx]
```

```
        nm = f"{p[1]}[{idx}]"
    else:
        print(f"Semantic error: '{p[1]}' not array.")
        v = 0
        nm = 'err'
    p[0] = {'place': nm, 'value': v}

def p_expr_func_call(p):
    'expr : ID LPAREN call_args RPAREN'
    temp = new_temp()
    tac.append(f"{temp} = call {p[1]}({', '.join(p[3])})")
    p[0] = {'place': temp, 'value': 0}

def p_empty(p):
    'empty :'

def p_error(p):
    if p:
        print(f"Syntax error at token '{p.value}', line: {p.lineno}")
    else:
        print("Syntax error at EOF")

parser = yacc.yacc()

test_code = '''
int arr[5], x, y; float f;
x = 5; y = 2; arr[1] = x + y * 2; f = 2.5;
print(arr[1]); print(f);

int sum(int a, int b) {
    int result;
    result = a + b;
    return result;
}
x = sum(x, y); print(x);

if (x == 9) { print(123); } else { print(321); }
while (x > 0) { x = x - 1; }
'''

print("--- TOKENS ---")
lexer.input(test_code)
for tok in lexer:
    print(tok)
print("\n--- PARSING ---")
parser.parse(test_code)
```

Listing 1: Mini Compiler Source Code

4 Results

4.1 Token Output

```
--- TOKENS ---
LexToken(INT,'int',2,1)
LexToken(ID,'arr',2,5)
```

```
LexToken(LSQUARE,'[',2,8)
LexToken(NUMBER,5,2,9)
LexToken(RSQUARE,']',2,10)
LexToken(COMMA,',',2,11)
LexToken(ID,'x',2,13)
LexToken(COMMA,',',2,14)
LexToken(ID,'y',2,16)
LexToken(SEMICOLON,';',2,17)
LexToken(FLOAT,'float',2,19)
LexToken(ID,'f',2,25)
LexToken(SEMICOLON,';',2,26)
LexToken(ID,'x',3,28)
LexToken(EQUALS,'=',3,30)
LexToken(NUMBER,5,3,32)
LexToken(SEMICOLON,';',3,33)
LexToken(ID,'y',3,35)
LexToken(EQUALS,'=',3,37)
LexToken(NUMBER,2,3,39)
LexToken(SEMICOLON,';',3,40)
LexToken(ID,'arr',3,42)
LexToken(LSQUARE,'[',3,45)
LexToken(NUMBER,1,3,46)
LexToken(RSQUARE,']',3,47)
LexToken(EQUALS,'=',3,49)
LexToken(ID,'x',3,51)
LexToken(PLUS,'+',3,53)
LexToken(ID,'y',3,55)
LexToken(TIMES,'*',3,57)
LexToken(NUMBER,2,3,59)
LexToken(SEMICOLON,';',3,60)
LexToken(ID,'f',3,62)
LexToken(EQUALS,'=',3,64)
LexToken(FNUMBER,2.5,3,66)
LexToken(SEMICOLON,';',3,69)
LexToken(PRINT,'print',4,71)
LexToken(LPAREN,'(',4,76)
LexToken(ID,'arr',4,77)
LexToken(LSQUARE,'[',4,80)
LexToken(NUMBER,1,4,81)
LexToken(RSQUARE,']',4,82)
LexToken(RPAREN,')',4,83)
LexToken(SEMICOLON,';',4,84)
LexToken(PRINT,'print',4,86)
LexToken(LPAREN,'(',4,91)
LexToken(ID,'f',4,92)
LexToken(RPAREN,')',4,93)
LexToken(SEMICOLON,';',4,94)
LexToken(INT,'int',6,97)
```

```
LexToken(ID,'sum',6,101)
LexToken(LPAREN,'(',6,104)
LexToken(INT,'int',6,105)
LexToken(ID,'a',6,109)
LexToken(COMMA,',',6,110)
LexToken(INT,'int',6,112)
LexToken(ID,'b',6,116)
LexToken(RPAREN,')',6,117)
LexToken(LBRACE,'{',6,119)
LexToken(INT,'int',7,125)
LexToken(ID,'result',7,129)
LexToken(SEMICOLON,';',7,135)
LexToken(ID,'result',8,141)
LexToken(EQUALS,'=',8,148)
LexToken(ID,'a',8,150)
LexToken(PLUS,'+',8,152)
LexToken(ID,'b',8,154)
LexToken(SEMICOLON,';',8,155)
LexToken(RETURN,'return',9,161)
LexToken(ID,'result',9,168)
LexToken(SEMICOLON,';',9,174)
LexToken(RBRACE,'}',10,176)
LexToken(ID,'x',11,178)
LexToken(EQUALS,'=',11,180)
LexToken(ID,'sum',11,182)
LexToken(LPAREN,'(',11,185)
LexToken(ID,'x',11,186)
LexToken(COMMA,',',11,187)
LexToken(ID,'y',11,189)
LexToken(RPAREN,')',11,190)
LexToken(SEMICOLON,';',11,191)
LexToken(PRINT,'print',11,193)
LexToken(LPAREN,'(',11,198)
LexToken(ID,'x',11,199)
LexToken(RPAREN,')',11,200)
LexToken(SEMICOLON,';',11,201)
LexToken(IF,'if',13,204)
LexToken(LPAREN,'(',13,207)
LexToken(ID,'x',13,208)
LexToken(EQ,'==',13,210)
LexToken(NUMBER,9,13,213)
LexToken(RPAREN,')',13,214)
LexToken(LBRACE,'{',13,216)
LexToken(PRINT,'print',13,218)
LexToken(LPAREN,'(',13,223)
LexToken(NUMBER,123,13,224)
LexToken(RPAREN,')',13,227)
LexToken(SEMICOLON,';',13,228)
```

```
LexToken(RBRACE,'}',13,230)
LexToken(ELSE,'else',13,232)
LexToken(LBRACE,'{',13,237)
LexToken(PRINT,'print',13,239)
LexToken(LPAREN,'(',13,244)
LexToken(NUMBER,321,13,245)
LexToken(RPAREN,')',13,248)
LexToken(SEMICOLON,';',13,249)
LexToken(RBRACE,'}',13,251)
LexToken(WHILE,'while',14,253)
LexToken(LPAREN,'(',14,259)
LexToken(ID,'x',14,260)
LexToken(GT,'>',14,262)
LexToken(NUMBER,0,14,264)
LexToken(RPAREN,')',14,265)
LexToken(LBRACE,'{',14,267)
LexToken(ID,'x',14,269)
LexToken(EQUALS,'=',14,271)
LexToken(ID,'x',14,273)
LexToken(MINUS,'-',14,275)
LexToken(NUMBER,1,14,277)
LexToken(SEMICOLON,';',14,278)
LexToken(RBRACE,'}',14,280)
```

4.2 Parsing and Output

```
--- PARSING ---
OUTPUT: 9
OUTPUT: 2.5
Syntax error at token 'int', line: 21
Semantic error: 'a' used before assignment.
Semantic error: 'b' used before assignment.
Syntax error at token 'return', line: 23
OUTPUT: 0
OUTPUT: 123
OUTPUT: 321

SYMBOL TABLE: [{'arr': <__main__.Array object at 0x000002F35381E410>, 'x': -1, 'y': 2
```

4.3 Three Address Code

```
x = 5
y = 2
t1 = y * 2
t2 = x + t1
arr[1] = t2
f = 2.5
```

```
print arr[1]
print f
t3 = a + b
result = t3
t4 = call sum(x, y)
x = t4
print x
t5 = x == 9
print 123
print 321
t6 = x > 0
t7 = x - 1
x = t7
```

5 References

- *PLY (Python Lex-Yacc) Documentation.* `https://www.dabeaz.com/ply/`
- Python Official Documentation. `https://docs.python.org/3/`
- Course lecture notes and textbooks on Compiler Design.